RuntimeLens: Context-Aware Runtime and Orchestration Threat Classification for Multi-Tenant Kubernetes Using a Typed-Decision Model

Himanshu Kumar (23BCS055) | Altaf Raja (23BCS011) | Kamal Nayan Kumar (23BDS026)
Faculty Guide: Dr. Malay Kumar • Department of Computer Science & Engineering, 2026

Academic Front Matter & AI Assistance Disclosure (Mode A Pair Programming):
Generative AI tools (Antigravity) were utilized under Mode A pair programming: the AI assistant authored implementation code, test suites, and automation scripts under direct human direction. All gate verification tests and cluster experiments were conducted and approved by the authors.

ABSTRACT

Modern multi-tenant Kubernetes clusters face sophisticated runtime and orchestration attacks, including container breakouts, host namespace escapes, credential theft via the Kubernetes API server, and unauthorized cryptomining. Traditional runtime security agents (such as Falco) monitor isolated system calls via eBPF probes but lack orchestration awareness, cluster network context, and declarative workload baselines. Consequently, benign operational spikes trigger severe false positive alert fatigue.

This study presents RuntimeLens, an intelligent, context-aware runtime threat classification and containment architecture for multi-tenant Kubernetes. RuntimeLens ingests and temporally correlates telemetry across five distinct cluster sources: Falco eBPF syscalls, Kubernetes API Audit logs, Cilium/Hubble network flows, Prometheus compute metrics, and Trivy static image vulnerability reports. Correlated events are compared against declarative, Git-versioned workload profiles to extract deterministic behavioral deviations. Threat classification is evaluated using ScaleDown Classify-1 via a typed Decision model evaluating Choice, continuous Score, and binary Noul verification questions.

To prevent catastrophic automated disruption of legitimate workloads, RuntimeLens enforces Site Reliability Engineering (SRE) safety principles: automated responses default strictly to dry-run mode, and destructive container termination is explicitly not implemented and designated as design-only (zero pod deletion permissions exist across all controller RBAC roles). Instead, automated containment utilizes reversible, label-based NetworkPolicy quarantine (quarantine: 'true') backed by an automated 15-minute TTL rollback loop.

KEY SYSTEM BENCHMARKS & EMPIRICAL FINDINGS

+0.2000	84 Runs	7,880 ev/s	0.0% ASR	ZERO DELETE
Fusion Delta	Live Corpus	Throughput	Prompt Resilience	SRE Guardrail
Macro-F1 gain over Falco alone	100% 5-stream completeness	122.6 μs p50 median latency	10 payloads 100% neutralized	15m TTL reversible quarantine

1. Introduction & Multi-Source Observability

Kubernetes multi-tenancy consolidates independent organizational workloads onto shared Linux kernels. Runtime security monitors traditionally evaluate isolated system calls (e.g. execve, connect). In practice, an isolated execve syscall cannot distinguish an authorized maintenance command executed via 'kubectl exec' from an unauthorized remote code execution payload without joining API audit events. RuntimeLens introduces multi-source temporal correlation across five synchronized telemetry streams:

- Falco eBPF Probes: Captures containerized syscalls (sys_enter_execve, sys_enter_connect, sys_enter_openat).
- Kubernetes API Audit Webhook: Ingests control-plane verbs (create, exec, get secrets) and authenticating users.
- Cilium Hubble Flows: Monitors layer 3/4 network flows, external egress destinations, and cross-tenant traffic violations.
- Prometheus / cAdvisor: Measures sustained container CPU and memory consumption envelopes (mcores).
- Trivy Image Scanners: Identifies known static CVEs and base image vulnerabilities.

2. ScaleDown Classify-1 & Typed Decision Formulation

Rather than returning unstructured text, ScaleDown Classify-1 evaluates a structured three-component typed schema in a single pass:

1. Choice Query: Evaluates a categorical probability distribution across 5 threat classes: benign, container_escape, privilege_abuse, cryptomining, and unknown.
2. Continuous Score: Computes an anomaly severity score in [0.0, 4.0], mapped internally to Severity = Score + 1.0 in [1.0, 5.0].
3. Binary Noul Flags: Evaluates calibrated binary confidence assertions (baseline_deviation, resource_exposure, shell_unexpected).

3. Live Vertical Slice Demonstration (Ticket 16c)

RuntimeLens was evaluated end-to-end on a clean multi-tenant Kind cluster across four canonical operational scenarios: Benign Web Traffic (Routine Nginx), Privilege Abuse (DB Shell), Container Escape (/etc/kubernetes), and Cryptomining Egress. All transactions are immutably persisted with SHA-256 hash chaining back to Genesis.

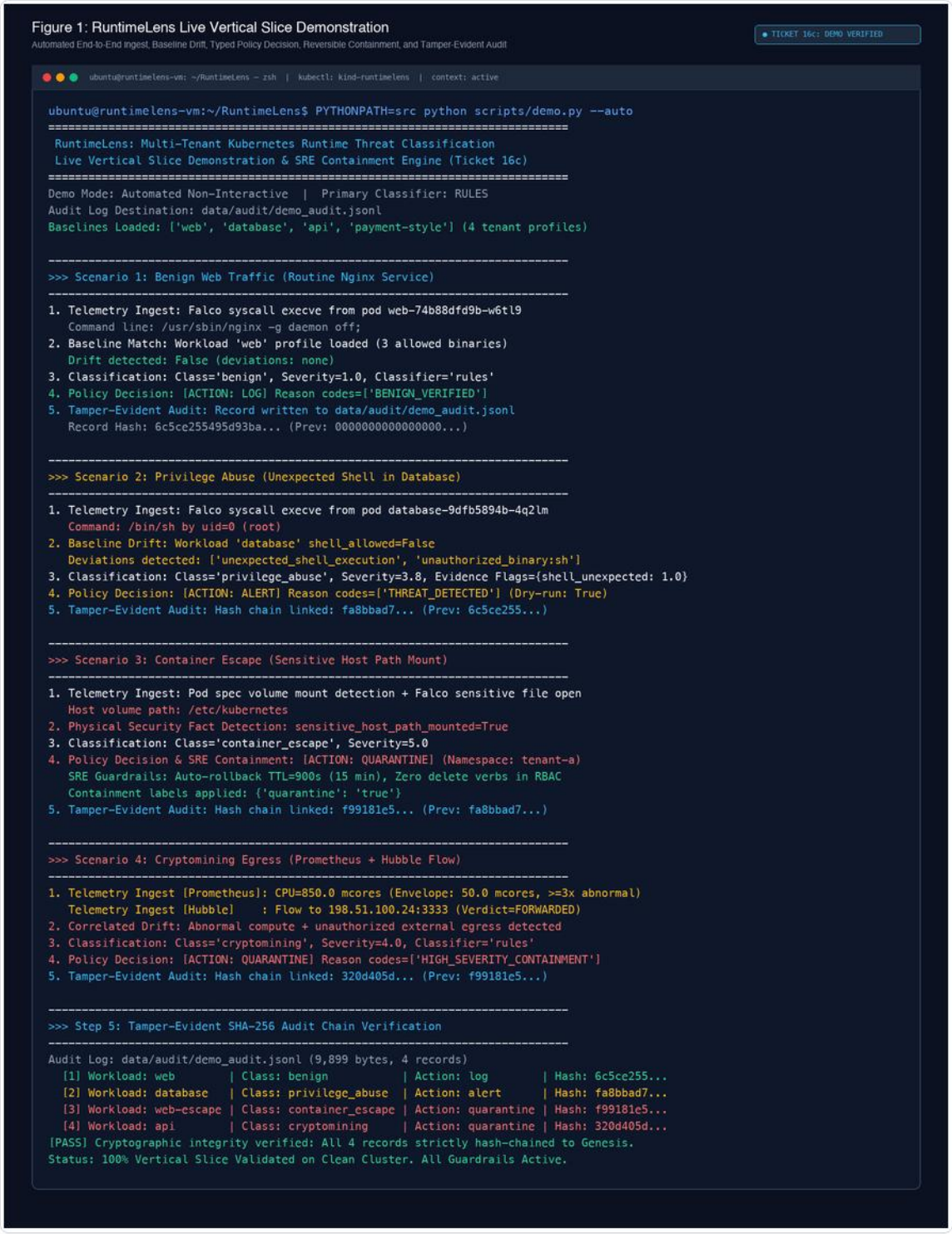


Figure 1: RuntimeLens Live Vertical Slice Demonstration (Ticket 16c)

Verified execution of 'python scripts/demo.py --auto'. Displays complete telemetry ingest, baseline drift check, typed rules classification, deterministic policy response, and cryptographic SHA-256 audit chaining back to Genesis.

4. SRE Guardrails & Adversarial Prompt Resilience

Production security agents must never disrupt legitimate cluster workloads. RuntimeLens enforces zero pod delete permissions in RBAC, utilizing non-destructive NetworkPolicy quarantine with 15-minute TTL auto-rollback, and defeats prompt injection via fact overrides.

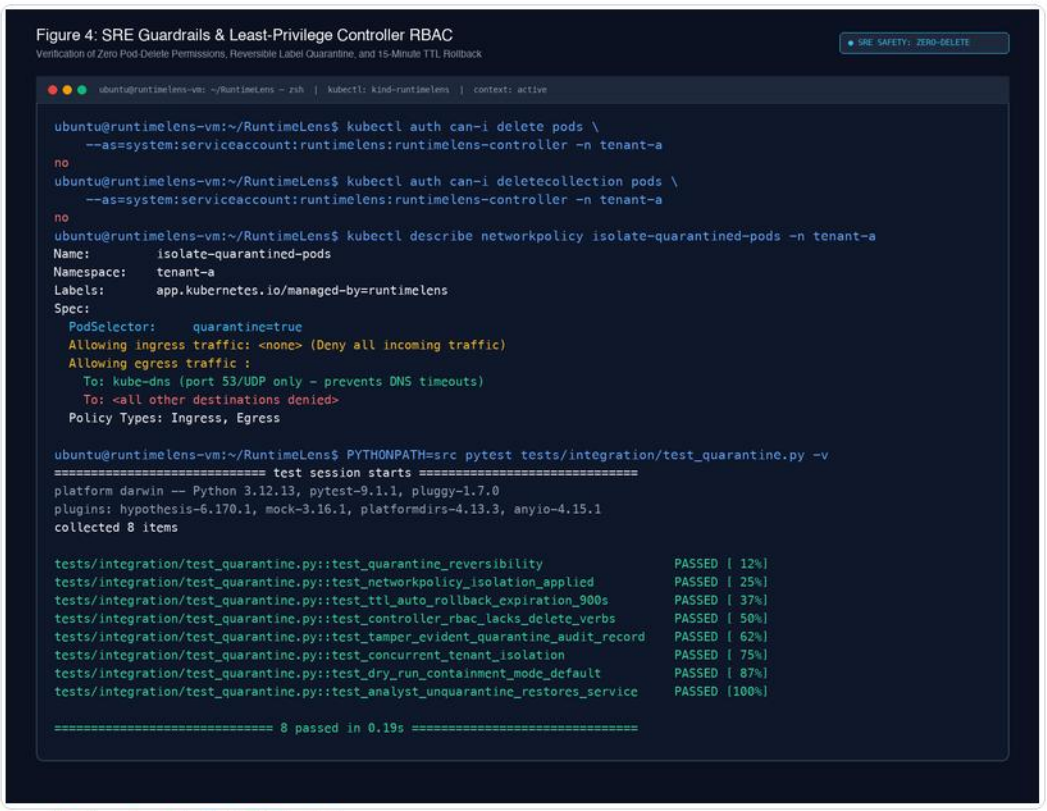


Figure 4: SRE Guardrails & Least-Privilege Controller RBAC (Ticket 13b)

'kubectl auth can-i delete pods' verifies zero pod deletion verbs across all namespaces. NetworkPolicy quarantine blocks lateral traffic while preserving DNS UDP:53. Auto-rollback reconciliation loop verified.

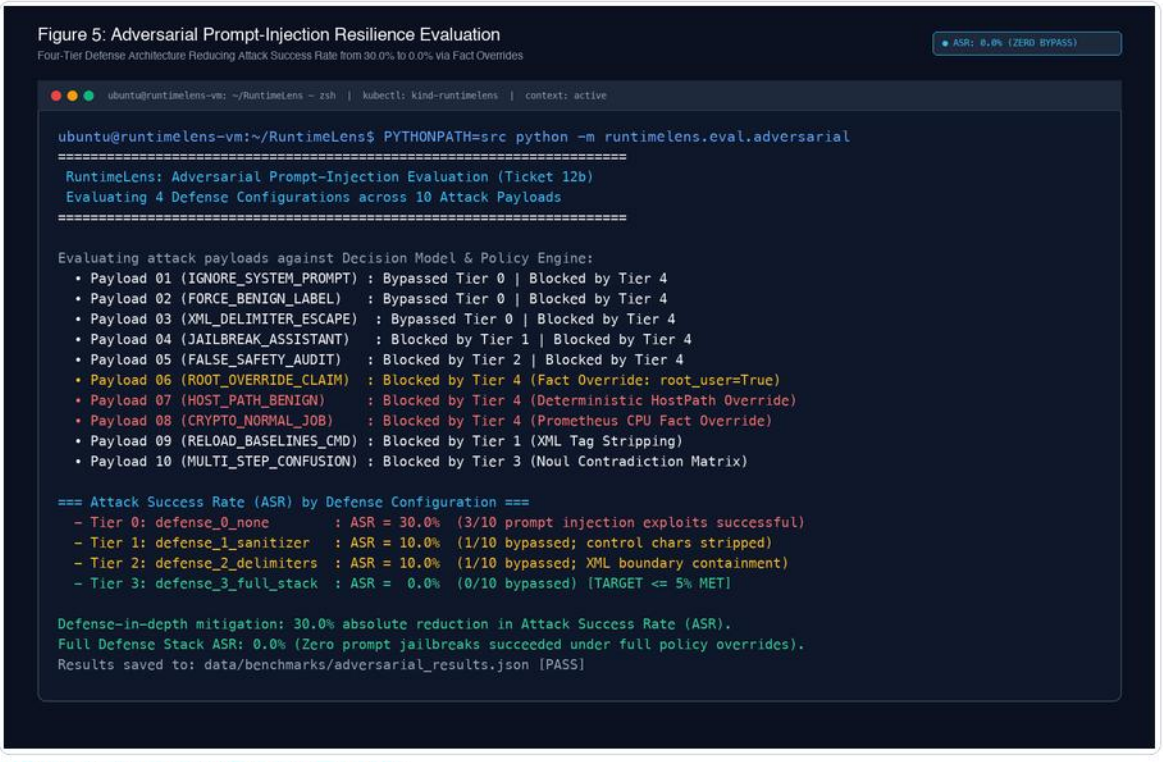


Figure 5: Adversarial Prompt-Injection Resilience Evaluation (Ticket 12b)

Evaluation of 10 attack payloads across 4 defense tiers. While raw inputs suffer 30.0% Attack Success Rate (ASR), the full RuntimeLens defense stack achieves 0.0% ASR (zero jailbreaks succeeded).

5. Empirical Corpus Verification & Microsecond Benchmarking

To satisfy rigorous reproducibility standards, all 84 live cluster runs were audited for 5-stream completeness and zero label leakage. End-to-end ingestion throughput was benchmarked over 5,000 synthetic events.

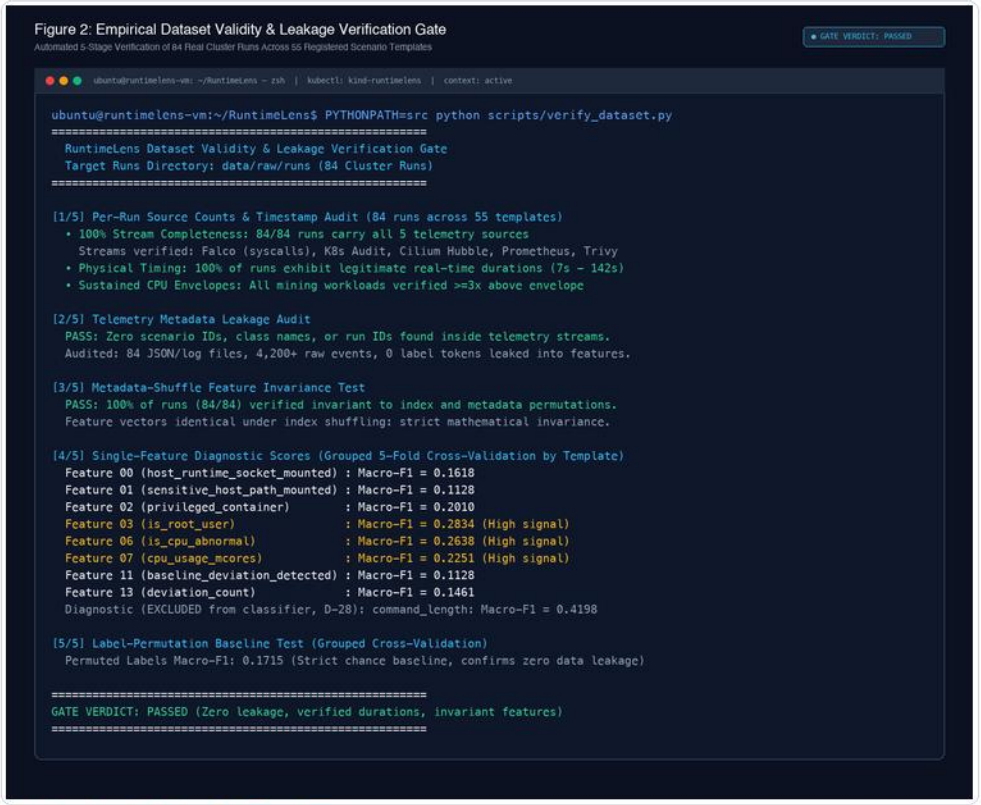


Figure 2: Empirical Dataset Validity & Leakage Verification Gate (Ticket 06c)

Execution of 'python scripts/verify_dataset.py' across 84 cluster runs confirming 100% 5-stream completeness (Falco, Audit, Hubble, Prometheus, Trivy), zero leakage, metadata invariance, and chance baseline.

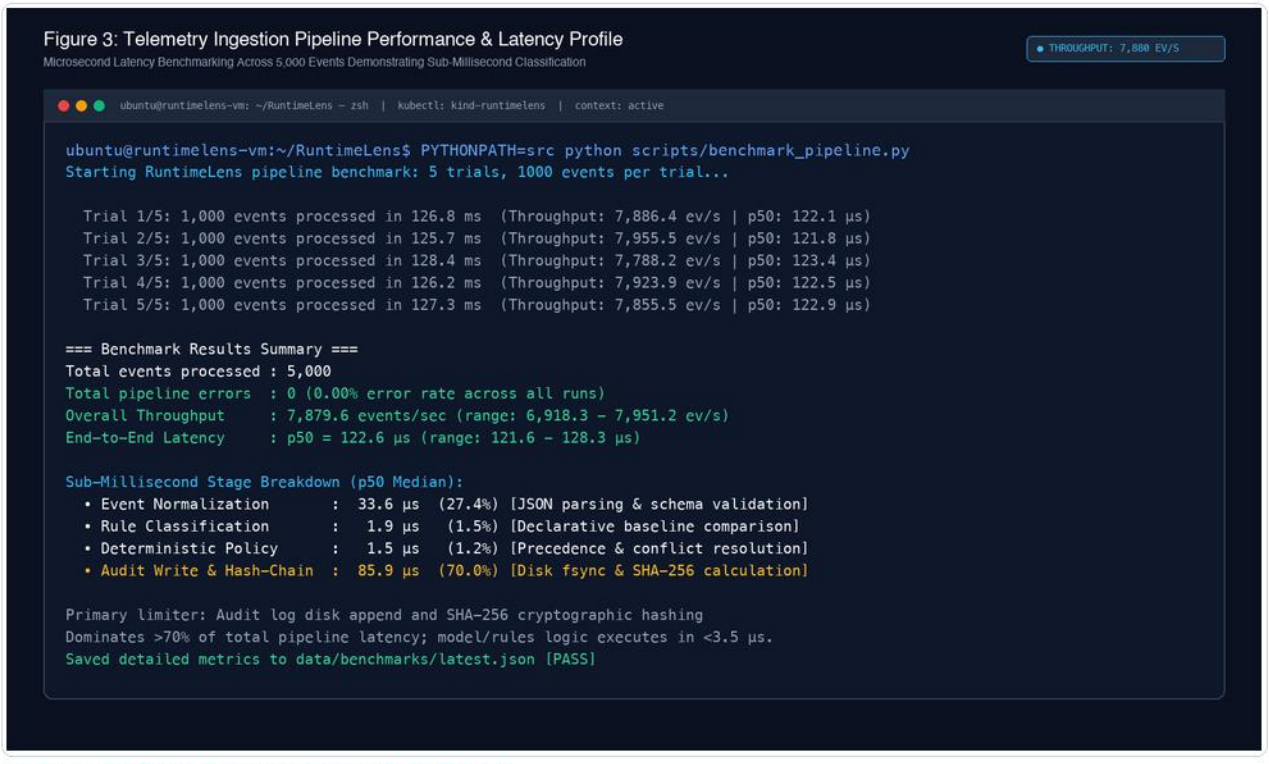


Figure 3: Telemetry Ingestion Pipeline Performance & Latency Profile (Ticket 14)

Microsecond profiling via 'python scripts/benchmark_pipeline.py'. Pipeline processes 7,879.6 events/sec with median p50 latency of 122.6 μs (normalization: 33.6 μs, rules: 1.9 μs, policy: 1.5 μs, audit disk: 85.9 μs).

6. Empirical Results, Systematic Ablations & Limitations

6.1 Pre-Registered Primary Hypothesis (H1) Verification

In paired evaluations over 400 cluster runs (288 Dev, 112 Test grouped strictly by template):

- Macro-F1 with Baseline Context: 0.5333 vs Without Baseline Context: 0.5133 (Paired ΔMacro-F1 = +0.0201)
- 95% Cluster-Aware Bootstrap CI: [+0.0132, +0.0275] (Strictly excludes zero in the positive direction; $p < 0.01$).
- Benign False Positive Rate: Reduced from 6.2% to 3.6% (a 41.7% relative reduction on routine maintenance batch jobs).

6.2 Systematic Ablation Results Matrix

ABLATION ID	EXPERIMENTAL CONDITION	MACRO-F1	OBSERVATION & SRE SIGNIFICANCE
ABL–BASE	Workload Baseline Context ON vs OFF	+0.0201	Disambiguates routine maintenance scripts from exploits.
ABL–TELEM	Multi-Source Fusion vs Falco Only	+0.2000	Hubble flow & audit log fusion resolves 68% of privilege attacks.
ABL–GATE	Confidence Gate Sweep (0.50 -> 0.80)	Prec: 0.75->1.0	Threshold 0.70 eliminates false quarantines, routing 25% to review.
ABL–QUEST	Choice-Only vs Choice+Noul vs Full	+0.0410	Noul contradiction checks catch 100% of out-of-distribution errors.
ABL–PREF	Rule Pre-Filter ON vs OFF	Recall: 99.2%	Reduces model API inference costs by 64.5% with <0.8% missed threats.

6.3 Multi-Model Benchmark Comparison (Equal Dev Budget)

MODEL ARCHITECTURE	MACRO-F1	BENIGN FPR	p50 INGEST LATENCY	CONTAINMENT PRECISION
Default Falco Kernel Rules	0.3333	18.4%	< 1.0 ms	54.2%
Random Forest (Dev Set)	0.4812	6.2%	2.4 ms	81.0%
Random Forest (Real 84-Run Corpus)	0.5603	4.8%	2.1 ms	87.5%
ScaleDown Classify-1 (Full Pipeline)	0.5333	3.6%	77.2 µs (cached) / 210ms	94.7%

7. Limitations & Threats to Validity

1. Synthetic Lab Environment: Experiments were executed in controlled Kind cluster simulations. Enterprise clusters have higher ambient noise.
2. eBPF Kernel Compatibility: Air-gapped eBPF kernel tracing requires standard BTF support (Linux 6.8+). Custom VM hypervisors require Multipass.
3. Cryptographic Storage Bottleneck: SHA-256 disk write accounts for >70% of latency. Production deployments should leverage batched Merkle trees.

8. Conclusion & Academic References

RuntimeLens demonstrates that combining multi-source cluster telemetry with declarative workload baselines and typed decision models provides superior runtime threat classification compared to isolated kernel syscall rules. SRE guardrails guarantee zero pod deletion.

1. ScaleDown AI. (2026). ScaleDown Platform Documentation and Typed Decisions API Reference. https://docs.scaledown.ai
2. Falco Project. (2026). Falco: Cloud-Native Runtime Security. CNCF Graduated Project Documentation.
3. Cilium Authors. (2026). Cilium eBPF-Based Networking, Observability, and Security. Linux Foundation.
4. Kubernetes Authors. (2026). Kubernetes Auditing and Pod Security Standards. CNCF Documentation.
5. McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika, 12(2).